

INSTITUT NATIONAL DES SCIENCES
APPLIQUEES DE TOULOUSE

5^{ème} Année SEE

**TECHNIQUES POUR
L'OPTIMISATION DE LA COMMANDE
ELECTRONIQUE**

**MACHINE LEARNING POUR LA
CONCEPTION ELECTRONIQUE**

**PRESENTATION DES METHODES
D'OPTIMISATION UTILISEES DURANT
LE TP**

Alexandre Boyer
alexandre.boyer@insa-toulouse.fr
www.alexandre-boyer.fr

Septembre 2025

Le TP de Machine learning pour la conception électronique vise à mettre en place un flot typique de simulation utilisant des méthodes de machine learning pour optimiser un problème de conception électronique complexe. Il s'appuie sur un cas d'étude, dédié à l'optimisation du placement de condensateurs de découplage sur un circuit imprimé afin de garantir une intégrité de puissance acceptable. Le but de cet enseignement n'a pas vocation à tester et comparer l'ensemble des méthodes de machine learning existante, mais plutôt d'en introduire plusieurs, qui sont plutôt populaires et adaptées au problème traité (ici, un problème d'optimisation). Dans ce TP, nous avons sélectionné trois approches différentes :

- Evolution différentielle
- Optimisation par essaims particuliers (*Particle Swarm Optimization*)
- Optimisation Bayésienne basée sur une régression par processus gaussien (krigeage)

Il est important de noter que les deux premières ne sont pas vraiment classées comme des méthodes de machine learning, mais sont plutôt des métaheuristiques d'optimisation. La dernière est une méthode d'optimisation basée sur une méthode de machine learning supervisé. Elles appartiennent cependant au domaine de l'intelligence artificielle et ont en commun de reposer sur des algorithmes dédiés à la résolution de problèmes complexes conçus sans instructions explicites sur les actions à mener et fonctionnant de manière automatique et sans intervention humaine.

Ce document vise à :

- donner la vue d'ensemble du flot d'optimisation utilisé pendant ce TP
- décrire succinctement le principe des méthodes (métaheuristiques et machine learning) utilisées durant ce TP, ainsi que leur paramétrage
- donner des exemples d'utilisation simples basés sur des bibliothèques Python open-source (les codes Python sont disponibles sur Moodle).

I. Flot d'optimisation utilisé

Le problème traité durant ce TP s'apparente à un problème d'optimisation non-linéaire contraint. En raison du nombre élevé de variables à optimiser (plusieurs dizaines) et du grand nombre d'optimum locaux, les algorithmes d'optimisation traditionnels reposant sur une descente de gradient ne sont pas adaptés. De plus, le problème traité nécessite l'utilisation d'un outil de simulation numérique dédié, potentiellement gourmand en temps de calcul. Des méta-heuristiques ou des méthodes d'optimisation basées sur l'apprentissage d'un métamodèle sont donc requis. Les méthodes mises en œuvre dans ce TP sont décrites dans la prochaine partie.

Quelles que soient la méthode utilisée, le flot d'optimisation restera le même. Son principe est illustré sur la Figure 1. Au préalable, **l'espace des solutions** (plage des valeurs pour les différentes variables à optimiser) et les **contraintes du problème** doivent être définis. Une fois l'algorithme d'optimisation ou d'apprentissage choisi, ses hyperparamètres doivent être réglés. Ces derniers vont influencer sur la convergence et la rapidité de la convergence. L'algorithme d'optimisation/d'apprentissage va chercher une combinaison de variables d'entrée à l'intérieur de l'espace des solutions correspondant au minimum d'une **fonction dite de cout**, qui correspond à la fonction à minimiser. Celle-ci doit être choisie judicieusement. La manière dont se fait cette recherche est pilotée par l'algorithme d'optimisation/d'apprentissage. Durant ce TP, nous utiliserons des fonctions existantes et fournies par des bibliothèques Python open-source.

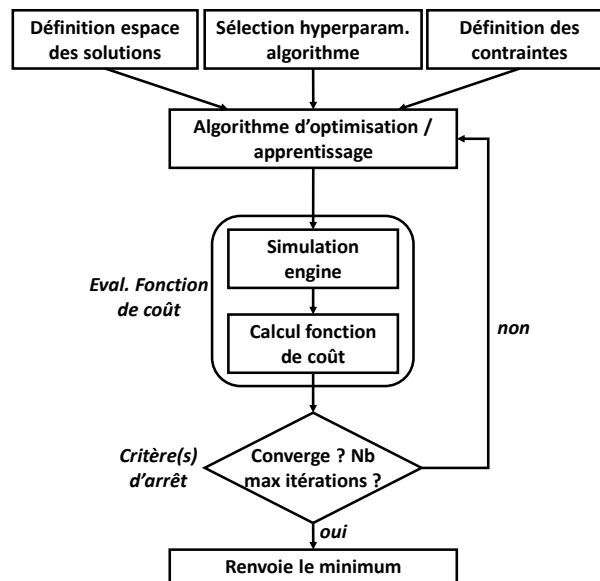


Figure 1 – Principe du flot d’optimisation mis en œuvre durant ce TP

La recherche de l’optimum est itérative. L’algorithme arrête la recherche selon un critère d’arrêt (lié à la convergence de la recherche et/ou un nombre maximal d’itérations). A chaque itération, l’algorithme fait appel à une autre fonction qui évalue la fonction de coût. A l’issue de cette recherche, la solution identifiée sera la combinaison de variables d’entrée qui minimise la fonction de coût tout en satisfaisant aux contraintes du problème.

La fonction de coût peut être simple à évaluer et directement calculée par le programme Python qui gère l’algorithme de recherche d’optimum. Dans ce TP, nous considérons un problème de conception électronique s’appuyant sur des outils de simulation numérique complexes et propriétaires. Pour l’utilisateur, il s’agit d’une « boîte noire » dont le fonctionnement interne est inaccessible. L’évaluation de la fonction de coût passe donc par un appel à cet outil de simulation externe, que nous appellerons **simulation engine**. L’interfaçage avec cet outil se fait via des fichiers d’entrée et de sortie, voire des fonctions dans le cas où l’outil de simulation propose une interface de pilotage.

Il est important de noter que le lancement du *simulation engine* engendre un temps d’exécution parfois non négligeable. Une manière de réduire ce temps d’exécution est l’utilisation d’un mode dit de **vectorisation**. Plutôt que d’envoyer une combinaison de variables d’entrée au *simulation engine*, ce mode consiste à envoyer N combinaisons de variables d’entrée et de collecter les résultats pour ces différentes combinaisons. Ce mode permet donc de diviser par N le temps lié au lancement du *simulation engine*. Cependant, tous les algorithmes d’optimisation/d’apprentissage ne s’y prêtent pas forcément. Les métaheuristiques évolutionnaires et PSO supportent ce mode, mais pas les méthodes de régression par krigeage.

Remarque : rien ne garantit à l’issue de l’exécution de ce flot que l’algorithme à identifier le minimum global du problème. Dans tous les cas, seuls l’utilisateur sera capable d’apprécier la pertinence de(s) la solution identifiée par l’algorithme.

II. Description succincte des méthodes utilisées durant le TP

1. Evolution différentielle (DE)

a. Principes

L'évolution différentielle est un algorithme évolutionnaire, dérivée de l'algorithme génétique, qui s'inspire de la sélection naturelle et de l'évolution pour identifier des solutions aux problèmes d'optimisation, plutôt que se baser sur le gradient de la fonction à optimiser. Ces méthodes sont aussi appelées métaheuristiques car elles ne font aucune hypothèse sur le problème à optimiser (considéré comme une boîte noire) et sont capables de faire une recherche de solution sur un large espace de solutions. Comparé à l'algorithme génétique, l'algorithme DE propose une approche simplifiée, faisant moins appel à des analogies avec la génétique, ne s'applique qu'à des valeurs réelles et s'avère généralement plus performante.

Le DE optimise un problème de manière stochastique, en conservant une population de solutions candidates et en créant de nouvelles en combinant celles qui existent déjà, puis en conservant la solution candidate qui minimise la fonction de coût. Différentes stratégies existent pour créer les nouveaux candidats : celles que nous utiliserons durant ce TP est basée sur la construction du nouveau candidat à partir du meilleur candidat de la génération précédente.

Pour formaliser la présentation de l'algorithme, soit $x \in \mathbb{R}^n$ une solution candidate appartenant à une population de candidats. Un candidat est donc un vecteur formé de n valeurs, où n est le nombre de variables à optimiser. Nous cherchons la solution qui minimise la fonction à valeurs réelles $f(x)$, la fonction de coût. L'algorithme est défini par 3 hyperparamètres :

- NP : la taille de la population ou nombre d'individus (par exemple, $10 \times n$)
- CR : la probabilité de mutation ou crossover probability ou recombination constant, compris entre 0 et 1.
- F : le coefficient de mutation ou differential weight, compris entre 0 et 2.

Initialisation : les NP individus sont initialisés aléatoirement sur l'espace des solutions. Leur fonction de coût est évaluée.

Boucle : tant que le critère d'arrêt n'est pas atteint :

1. Chaque individu x_{old} de la population va évoluer de la manière suivante :

$$x' = x_{best} + F \times (x_{rand0} - x_{rand1})$$

Où le vecteur x' contient les prochaines valeurs probables de l'individu, x_{best} le meilleur candidat de la dernière génération (celui qui minimise la fonction de coût), x_{rand0} et x_{rand1} sont deux candidats de la dernière génération sélectionnés aléatoirement.

2. Pour chaque entrée $n^o i$ du vecteur x ($i \in [1; n]$), on tire un nombre aléatoire r_i compris entre 0 et 1. Si $r_i < CR$, alors l'entrée $n^o i$ du prochain candidat x_{new} prend l'entrée $n^o i$ du vecteur x' . Sinon, il conserve l'ancienne valeur (l'entrée $n^o i$ du candidat x_{old}).

3. On calcule la fonction de coût des nouveaux individus x_{new} de la population.

4. Si $f(x_{new}) < f(x_{old})$, alors x_{new} remplace x_{old} .

Fin : l'algorithme renvoie l'individu présentant la fonction de coût la plus faible.

Il n'y a pas de choix à priori optimaux pour les hyperparamètres, mais il est possible d'analyser leur effet. Augmenter la taille de la population permet d'accroître les chances de trouver l'optimum global, au prix d'un temps de calcul plus long. Augmenter le coefficient de mutation F accroît le rayon de la recherche mais au prix d'une convergence plus lente. Même chose si on augmente la probabilité de mutation CR . De plus, de très fortes valeurs de CR et F peuvent induire une instabilité dans la

recherche de solutions. Jouer sur ces deux paramètres revient à favoriser l'exploration ou l'exploitation de l'espace des solutions.

b. Fonction `Scipy.Differential_evolution`

Durant ce TP, nous utiliserons la fonction `differential_evolution` de la librairie Scipy pour mettre en œuvre l'algorithme de DE. Cette librairie est directement intégrée dans la distribution Anaconda. Une description détaillée de la fonction peut être trouvée ici : https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.differential_evolution.html

Voici une description succincte des principaux arguments d'entrée de cette fonction :

`differential_evolution(func, bounds, args=(), strategy='best1bin', maxiter=1000, popsize=15, tol=0.01, mutation=(0.5, 1), recombination=0.7, rng=None, callback=None, disp=False, polish=True, init='latinhypercube', atol=0, updating='deferred', workers=1, constraints=(), x0=None, *, integrality=None, vectorized=False, seed=None)`

- `func` : la fonction appelée pour lancer l'évaluation de la fonction de coût.
- `bounds` : vecteur donnant les bornes de l'espace des solutions.
- `strategy` : la stratégie d'évolution différentielle mise en œuvre. On pourra conserver la stratégie 'best1bin'.
- `maxiter` : le nombre maximal d'itérations ou de générations. Si le critère de tolérance n'est pas atteint, l'algorithme sera exécuté `Maxiter+1` fois.
- `popsize` : définit la taille de la population (NP) = `popsize` × nombre de variables à optimiser.
- `tol` : le critère d'arrêt basé sur la tolérance relative pour la convergence
- `atol` : le critère d'arrêt basé sur la tolérance absolue pour la convergence
- `mutation` : les valeurs min et max du coefficient de mutation F. La fonction peut changer aléatoirement cette valeur.
- `recombination` : la probabilité de mutation CR.
- `polish` : à l'issue des `maxiter+1` itérations, une optimisation supplémentaire peut être lancée pour affiner le résultat trouvé. Ce n'est pas forcément conseillé dans notre TP, car le temps de calcul peut être ralenti.
- `Init` : définit la manière dont les valeurs initiales des candidats sont échantillonnées. L'échantillonnage de type hypercube latin est un choix pertinent, puisqu'il cherche à maximiser la couverture de l'espace des solutions.
- `constraints` : définition des contraintes linéaires et non-linéaires du problème (voir exemple dans la partie III.1)
- `vectorized` : pour activer le mode de vectorisation.

La fonction renvoie un objet de type `OptimizeResult`, dont les détails sont donnés ici : <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.OptimizeResult.html#scipy.optimize.OptimizeResult>

2. Optimisation par essais particulaires ou Particle Swarm Optimization (PSO)

a. Principes

L'algorithme d'optimisation par essais particulaires (PSO) est une métaheuristique d'optimisation bio-inspirée, à l'instar de l'algorithme DE. Bien qu'utilisant une terminologie et un algorithme différents, leurs principes restent assez proches. La méthode PSO consiste en une population (ou essaim) de solutions candidates (appelées particules) dont les coordonnées dans un espace à n

dimensions correspondent aux valeurs prises par les n variables à optimiser. A chaque itération, les particules sont déplacées en modifiant leurs positions et leurs vecteurs vitesse (vélocité) selon un loi mathématique simple. Le mouvement de chaque particule est non seulement influencé par la meilleure position qu'elle a pu rencontrer (celle qui minimise la fonction de coût), mais aussi par les meilleures positions rencontrées par les autres particules de l'essaim. Cela garantit que les particules convergent toutes vers un optimum (qui est espéré global, même si rien ne le garantit). Cette méthode s'inspire ainsi de vols d'oiseaux ou de bancs de poissons pour localiser la nourriture, où chaque individu exploite ce qu'il a appris via son mouvement local (via un paramètre cognitif) et ce qu'il perçoit des autres individus (via un paramètre social). Dès qu'un individu localise une source de nourriture, les autres vont chercher à reproduire son comportement. Ainsi, en jouant sur ces paramètres cognitif et social, les individus vont pouvoir chercher efficacement la solution dans l'espace de recherche.

Pour formaliser la présentation de l'algorithme, soit $x \in \mathbb{R}^n$ une solution candidate appartenant à une population de particules. Une particule est donc un vecteur formé de n valeurs, où n est le nombre de variables à optimiser. Nous cherchons la solution qui minimise la fonction à valeurs réelles $f(x)$, la fonction de coût. L'algorithme est défini par 4 hyperparamètres :

- NP : la taille de l'essaim ou nombre de particules (par exemple, $10 \times n$)
- w : est le facteur d'inertie (généralement compris entre 0.8 et 1.2)
- C1: le paramètre cognitif.
- C2 : le paramètre social. Il est préférable d'avoir $C1+C2 < 4$.

Chaque particule, repéré par un indice i , conserve la mémoire de sa meilleure position P_{best_i} .

Initialisation : les positions $P_i(0)$ des NP particules sont initialisées aléatoirement sur l'espace des solutions, ainsi que leurs vélocités initiales $V_i(0)$. La meilleure position de chaque particule, P_{best_i} , est initialisée avec $P_i(0)$. On évalue la meilleure position prise de l'essaim g_{best} en évaluant la fonction de coût de chaque particule.

Boucle : tant que le critère d'arrêt n'est pas atteint, on itère ($t \leftarrow t+1$) en modifiant la position et la vélocité de chaque particule :

1. Pour chaque vecteur particule et pour chaque entrée de ce vecteur :
 - On tire aléatoirement entre 0 et 1 deux valeurs notées r_1 et r_2 .
 - On met à jour la vélocité de chaque particule d'indice i selon l'équation suivante :

$$V_i(t+1) = w \times V_i(t) + C_1 r_1 (P_{best_i} - P_i(t)) + C_2 r_2 (g_{best} - P_i(t))$$
 - On met à jour la position de chaque particule d'indice i :

$$P_i(t+1) = P_i(t) + V_i(t+1)$$

2. On met à jour la meilleure position prise par chaque particule d'indice i :

$$\text{Si } f(P_i(t+1)) < f(P_{best_i}) \rightarrow P_{best_i} = P_i(t+1)$$

3. On met à jour la meilleure position prise par l'essaim :

$$\text{Si } f(P_{best_i}) < f(g_{best}) \rightarrow g_{best} = P_{best_i}$$

Fin : l'algorithme renvoie la meilleure position de l'essaim g_{best} .

Là encore, le choix des hyperparamètres influe sur la convergence de la méthode. Le facteur d'inertie définit la capacité d'exploration de chaque particule. Une grande valeur du facteur d'inertie induit une grande amplitude de mouvement et donc une plus grande exploration globale. Une faible valeur induira une exploration plus locale. Une valeur trop importante (> 1.2) risque de compromettre la convergence de l'algorithme. Augmenter $C1$ tend à accroître l'instinct de conservation de chaque particule, tandis qu'augmenter $C2$ tend à inciter les particules à suivre les particules voisines. Ainsi,

augmenter C1 et diminuer C2 tend à favoriser l'exploration de l'espace. Inversement, augmenter C2 et diminuer C2 favorise l'exploitation.

b. Fonction skopt.PSO

Durant ce TP, nous utiliserons la fonction PSO de la librairie Scikit_opt (<https://scikit-opt.github.io/>) pour mettre en œuvre l'algorithme PSO. Cette librairie n'est pas nativement présente dans la distribution Anaconda. Vous pouvez l'installer depuis un IDE Python (comme Spyder) à l'aide de la commande suivante : `pip install scikit-opt`.

Une description de la fonction peut être trouvée ici : <https://scikit-opt.github.io/scikit-opt/#/en/README?id=3-psoparticle-swarm-optimization>

Voici une description succincte des principaux arguments d'entrée de cette fonction :

`PSO(func, n_dim, pop=50, max_iter=200, lb, ub, w=0.8, c1=0.5, c2=0.5, constraint_ueq)`

- `func` : la fonction appelée pour lancer l'évaluation de la fonction de coût.
- `n_dim` : la dimension du problème, c'est-à-dire le nombre de variables à optimiser
- `pop` : définit la taille de la population ou de l'essaim
- `maxiter` : le nombre maximal d'itérations ou de générations.
- `lb, ub` : vecteurs donnant les limites basses et hautes de l'espace de recherche
- `w` : le facteur d'inertie.
- `c1` et `c2` : les paramètres cognitif et social.
- `Constraint_ueq` : définition des contraintes non-linéaires du problème (voir exemple dans la partie III.2)

La fonction propose aussi un mode de vectorisation. Pour cela, appelez la fonction suivante :

```
mode = 'vectorization'  
set_run_mode(func, mode)
```

3. Bayesian optimization using Gaussian Processes Regression (Kriging)

a. Principes

L'algorithme d'optimisation bayésienne (BO) est une méthode efficace dès qu'il s'agit d'explorer un problème complexe et coûteux (en temps de calcul). Cet algorithme permet un échantillonnage adaptatif de l'espace des solutions, en positionnant les futurs points là où ils seront le plus susceptibles de réduire l'incertitude du modèle. L'algorithme proposé utilise l'approche de régression par processus gaussien (krigeage) pour établir un métamodèle de la fonction de coût et identifier le prochain point d'échantillonnage selon une fonction d'acquisition. Selon les hyperparamètres sélectionnés, la fonction d'acquisition favorise soit l'exploration (en ajoutant des points où l'incertitude estimée est maximale) ou l'exploitation (en ajoutant des points là où la moyenne estimée est minimale). Le flot typique de l'optimisation bayésienne est décrit sur la figure ci-dessous.

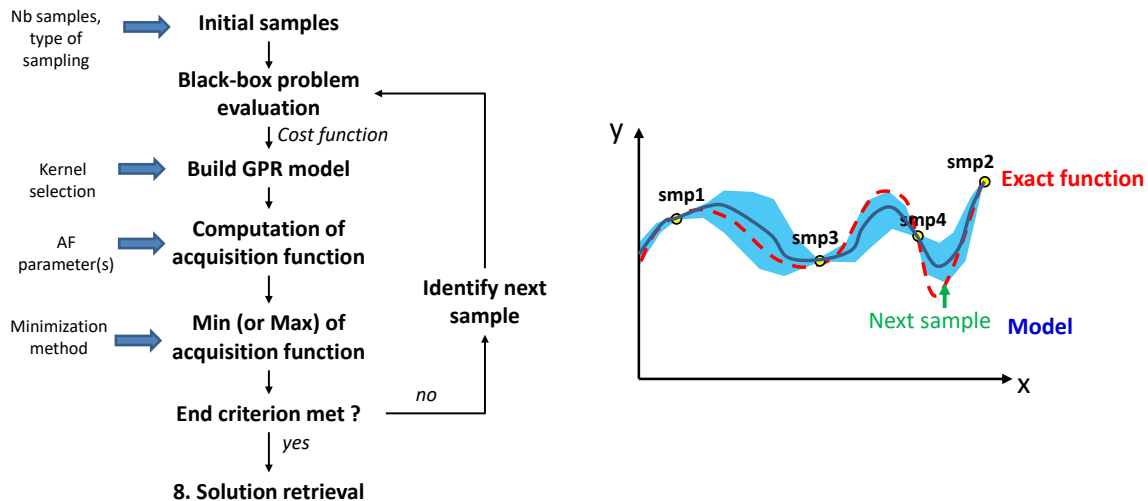


Figure 2 – Flot typique de l’optimisation bayésienne (à gauche) et principe de l’échantillonnage adaptatif (à droite)

Le prochain point d’échantillonnage est sélectionné là où la fonction d’acquisition $a(x)$ est minimale. Plusieurs fonctions d’acquisition courantes existent :

- Lower confidence bound (LCB) : $a(x;\lambda)=m(x)-\lambda.\sigma(x)$, où $m(x)$ et $\sigma(x)$ sont les espérances et les écart-types prédits par le modèle GPR et λ un paramètre choisi par l’utilisateur. Cette fonction d’acquisition est une simple combinaison linéaire entre l’espérance et la variance, pondérée par le terme λ . Un fort λ favorise l’exploration, tandis qu’une faible valeur favorise l’exploitation.
- Probability of Improvement (PI) : comme l’estimateur est un processus gaussien, la probabilité que la valeur en un point x soit plus faible que la valeur aux précédents meilleurs points, il est possible d’établir en tout point la probabilité de trouver un meilleur point. C’est pourquoi on parle d’amélioration. Le prochain point d’échantillonnage est sélectionné là où cette probabilité est maximale. Un hyperparamètre $\zeta > 0$ est introduit pour indiquer quelle amélioration de la PI est attendu. Une forte valeur de ζ favorise l’exploitation.
- Expected Improvement (EI) : elle est similaire à la précédente, hormis que l’espérance mathématique de l’amélioration qu’apporterait un échantillonnage en un point est estimé. Le prochain point d’échantillonnage est sélectionné là où cette espérance est maximale. De même, un hyperparamètre $\zeta > 0$ est introduit pour indiquer quelle amélioration de l’EI est attendu. Une forte valeur de ζ favorise l’exploitation.

b. Fonction `gp_minimize`

Durant ce TP, nous utiliserons la fonction `gp_minimize` de la librairie `Scikit_optimize` (https://scikit-optimize.github.io/stable/modules/generated/skopt.gp_minimize.html) pour mettre en œuvre l’algorithme BO. Cette librairie n’est pas nativement présente dans la distribution Anaconda. Vous pouvez l’installer depuis un IDE Python (comme Spyder) à l’aide de la commande suivante : `pip install scikit-optimize`.

Voici une description succincte des principaux arguments d’entrée de cette fonction :

```
gp_minimize(func, bounds, acq_function, kappa, xi, n_calls, n_initial_points, initial_point_generator,
acq_optimizer, n_restarts_optimizer, random_state)
```

- `func` : la fonction appelée pour lancer l’évaluation de la fonction de coût.
- `bounds` : vecteur donnant les bornes de l’espace des solutions.

- `acq_function` : définit la fonction d'acquisition ("LCB", "EI", "PI""gp_hedge","Elps", "PIps")
- `kappa` et `xi` : les hyperparamètres pour les fonctions d'acquisition LCB et EI/PI respectivement
- `n_calls` : le nombre maximal d'itérations de la fonction `func`
- `n_initial_points` : le nombre d'évaluation initiale de la fonction `func` avant que l'estimation par GPR ne soit lancée
- `initial_point_generator` : définit le type de générateurs pour les points d'évaluation initiale (`n_initial_points`) (« random","sobol","halton","hammersly","lhs")
- `acq_optimizer` : définit la méthode employée pour chercher le minimum de la fonction d'acquisition ("sampling" or "lbfgs")
- `n_restarts_optimizer` : dans le cas où la minimisation de la fonction d'acquisition est effectuée par la méthode L-BFGS, ce paramètre définit le nombre de redémarrage de la méthode. Attention, augmenter ce paramètre ralentit le processus d'optimisation.
- `random_state` : définit la séquence de génération pseudo-aléatoire pour le tirage des points initiaux. En spécifiant une valeur, on garantit un résultat reproductible.

Par rapport aux deux autres méthodes, on remarque qu'on ne peut pas définir de contraintes de type inégalités. En outre, l'approche BO étant basée sur un échantillonnage adaptatif, aucune vectorisation n'est possible.

III. Exemples de mise en œuvre

Afin de prendre en main les fonctions d'optimisation, plusieurs programmes simples sont fournis. Le problème traité est la recherche du minimum de la fonction à 2 dimensions :

$$f(x; y) = \sqrt{\sqrt{(x - x_0)^2 + (y - y_0)^2}}$$

où x_0 et y_0 sont deux paramètres réglables par l'utilisateur et correspondent au minimum de la fonction. Dans les exemples fournis, x_0 et y_0 sont réglés à +1 et -1 respectivement. La Figure 3 présente la surface de réponse de cette fonction.

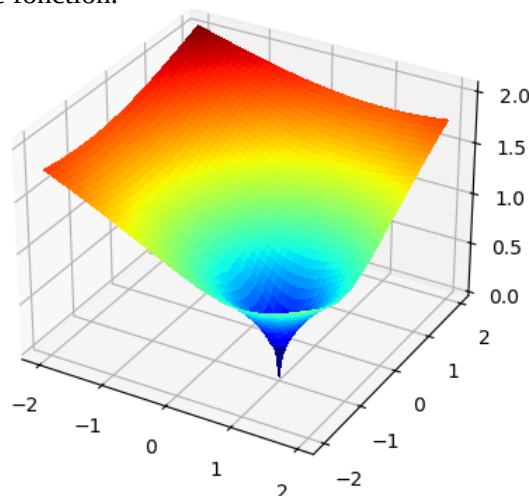


Figure 3 – Fonction à optimiser : le minimum se situe en $(x = +1 ; y = -1)$

Dans ces exemples, le critère d'arrêt est uniquement défini par un nombre maximal d'itérations. L'espace des solutions est borné à $[-2 ; +2]$ pour les 2 variables à optimiser. Une contrainte non-linéaire est ajoutée : $0 < \sqrt{x^2 + y^2} < 2$.

Malgré la simplicité de la fonction à évaluer (qui pourrait être directement évaluée par le programme Python), celle-ci est évaluée par une application .exe externe, afin d'illustrer la manière de piloter le *simulation engine* depuis le programme Python. Pour cela, nous utilisons l'application Calcul_Fonction_Obj.exe, fourni dans le dossier zippé script_prise_en_main.zip. Celle-ci attend les 4 arguments lors de son appel :

Calcul_Fonction_Obj.exe NbCalcul Fichier_individus.txt Fichier_resultats.txt Valeur_x0 Valeur_y0

- Le premier argument (NbCalcul) est un entier qui indique le nombre d'individus passé à l'application, donc le nombre de fois où la fonction devra être évaluée. Si NbCalcul > 1, alors le mode vectorisation est activé. Dans ce cas, NbCalcul = NP, la taille de la population.
- Le second argument (Fichier_individus.txt) passe les coordonnées (x ;y) des NP individus d'une génération. Le fichier est structuré sous le format d'une matrice avec NP lignes et 2 colonnes (x y), les 2 colonnes étant séparées par un espace.
- Le troisième argument (Fichier_resultats.txt) contient les résultats de l'évaluation de la fonction par l'application. Les NP résultats apparaissent sous la forme d'une matrice colonne.
- Les deux derniers arguments donnent les valeurs des termes x_0 et y_0 de la fonction, c'est-à-dire les coordonnées du minimum attendu.

1. Evolution différentielle (Scipy.Differential_evolution)

Le programme exemple est : DE_Fonction_ext_Run_Vectorized.py. Il est configuré avec :

- Une population de $20 \times 2 = 40$ individus
- 20 itérations maximales
- Probabilité de mutation = 0.7
- Coefficient de mutation = 0.5

La contrainte non-linéaire est passée de la manière suivante : définition d'une fonction qui évalue la fonction non-linéaire associée, d'une borne basse et d'une borne haute, qui sont passées à un objet de type NonLinearConstraint (plus de détails ici : <https://docs.scipy.org/doc/scipy-1.15.2/reference/generated/scipy.optimize.NonlinearConstraint.html>).

```
#définition de contraintes nonlinéaires : par ex.  $0 \leq \sqrt{x^2+y^2} \leq 2$ 
NbContraintesNonLin = 1
lowerBoundNonLinearConstraints = 0*np.ones(NbContraintesNonLin)
upperBoundNonLinearConstraints = 2*np.ones(NbContraintesNonLin)
```

```
#déf de la fonction de contraintes non-linéaires.
```

```
def MyNonlinearConstraint(p):
    x, y = p
    if np.size(x) == 1 :
        res = np.sqrt(x**2+y**2)
    else:
        taille_x = np.shape(x)
        #print("Taille x = "+str(taille_x[0]))
        Nb_calcul = taille_x[0]
        res = np.zeros((1,Nb_calcul))
        for ii in range(0,Nb_calcul,1):
            res[0,ii] = np.sqrt(x[ii]**2+y[ii]**2)
    return res
```

```
nonlinearConstraints = NonlinearConstraint(MyNonlinearConstraint, lowerBoundNonLinearConstraints,
upperBoundNonLinearConstraints )
```

Au bout de 9.5 s, l'algorithme identifie la solution suivante, qui est très proche du résultat attendu : [1.000e+00 ; -1.000e+00], avec une fonction de coût minimale de 0.00494807450928616. La figure ci-dessous présente la convergence de l'algorithme et les positions prises par le meilleur individu à chaque itération. On vérifie que l'algorithme converge progressivement vers la bonne solution. Les premiers minima trouvés sont relativement éloignés de la solution, mais se rapproche rapidement de celle-ci.

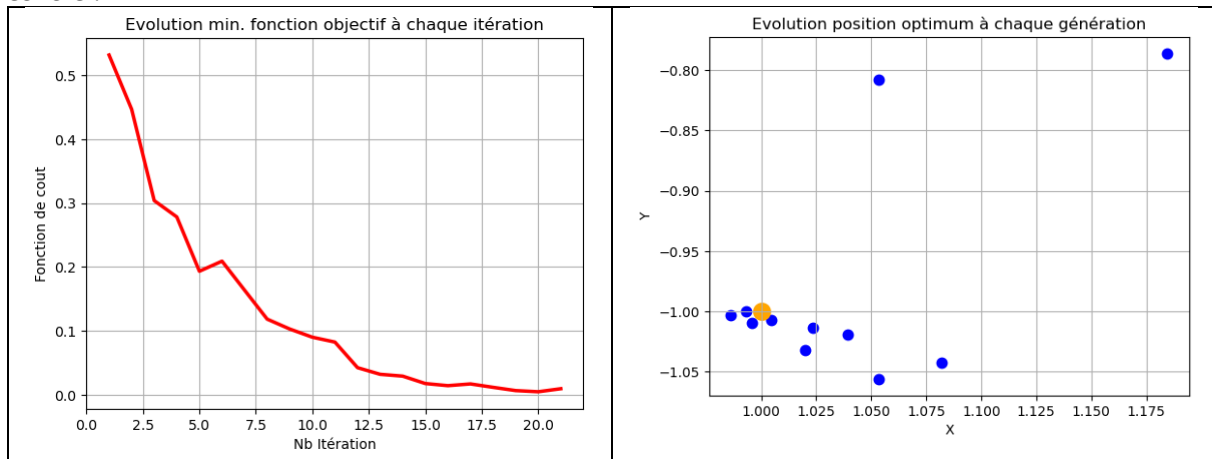


Figure 4 – Résultat algorithme DE : convergence (à gauche) et positions prises par le meilleur individu à chaque itération (en orange, la solution) (à droite).

On remarque aussi que lors des premières itérations, la taille de la population n'est pas exactement de 40, mais peut être comprise entre 20 et 30. Ce n'est plus le cas lors des dernières itérations. Cela vient de la contrainte utilisée ($x^2 + y^2 < 2$). L'algorithme DE de Scipy élimine les points qui ne satisfont pas à ce critère. Quand l'algorithme converge vers la bonne solution, les candidats qu'il génère satisfont de mieux en mieux cette contrainte.

2. Particle Swarm Optimization (skopt.PSO)

Le programme exemple est : PSO_Fonction_ext_Run_Vectorized.py. Il est configuré avec :

- Une population de 30 particules
- 25 itérations maximales
- Un facteur d'inertie = 0.8
- Des paramètres cognitif et social = 0.5

La contrainte non-linéaire est passée de la manière suivante :

```
#définition d'une contrainte d'inégalité :
# sqrt(x2+y2) <= R
#Ca se traduit sous la forme : sqrt(x2+y2) - R <= 0
constraint_ueq = [
    lambda x: np.sqrt(x[0]**2 + x[1]**2)-2
]
```

Au bout de 11 s, l'algorithme identifie la solution suivante, qui est très proche du résultat attendu : [1.00981394 ; -0.9967156], avec une fonction de coût minimale de 0.10172976171989. La figure ci-dessous présente la convergence de l'algorithme et les positions prises par le meilleur individu à chaque itération. On vérifie que l'algorithme converge progressivement vers la bonne solution. Les premiers minima trouvés sont relativement éloignés de la solution, mais se rapproche rapidement de celle-ci.

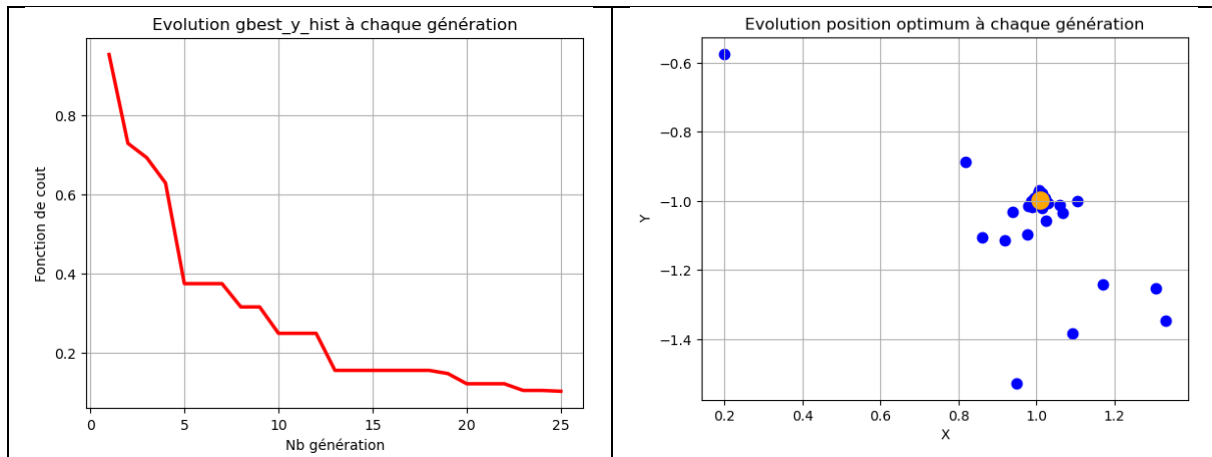


Figure 5 – Résultat algorithme PSO : convergence (à gauche) et positions prises par le meilleur individu à chaque itération (en orange, la solution) (à droite).

3. Optimisation bayésienne (Scikit-optimize.gp_minimize)

Le programme exemple est : BO_Fonction_ext_Run.py. Il est configuré avec :

- Un nombre d'itérations max de 80
- 20 itérations initiales, sélectionnées par un générateur de type LHS
- Une fonction d'acquisition de type LCB avec un paramètre kappa = 3
- La méthode L-BFGS pour identifier le minimum de la fonction d'acquisition (4 redémarrage de la recherche)

La contrainte non-linéaire est passée sous la forme d'une pénalité (= 4) ajoutée à la fonction de coût :

```
#ajout de la containte : on pénalise la fonction objectif si la contrainte
#n'est pas respectée. Pour cela, on fixe arbitrairement une valeur max
#(plus grande que le minimum recherché) quand la contrainte n'est pas respectée.
distance = np.sqrt((x-X0)**2+(y-Y0)**2)
if distance > Rmax: #violation de la contrainte
    Fonction_cout = penalite #attention à ne pas mettre une valeur trop grande, qui dégrade le
    #modèle GPR extrait.
```

Au bout de 51 s (le temps de calcul est plus long que les deux autres approches car on ne profite pas de la vectorisation, mais le nombre d'itérations de l'outil de simulation est nettement plus faibles), l'algorithme identifie la solution suivante, qui est très proche du résultat attendu : [0.9997521595025813 ; -0.9768736010002019], avec une fonction de coût minimale de 0.152078029272925. La figure ci-dessous présente la convergence de l'algorithme et les positions prises par le meilleur individu à chaque itération. On vérifie que l'algorithme converge progressivement vers la bonne solution. Au cours de l'exécution du programme, la fonction de coût affiche parfois 4 : il s'agit de la valeur de pénalité, qui a été sélectionné à une valeur suffisamment grande pour ne pas être considérée comme un minimum intéressant.

La figure présente aussi le tracé en 2D de la fonction de coût estimée par le GPR : celle-ci présente un minimum unique centré sur le point (1 ; -1). Les points d'échantillonnage sont aussi ajoutés à la figure. Ceux-ci sont plutôt regroupés autour du minimum, indiquant qu'au fur et à mesure du calcul, l'algorithme a ajouté des points d'échantillonnage autour du minimum (phase d'exploitation). Il en a aussi ajouté sur les bords, sans doute pendant la phase initiale et pour les besoins de l'exploration de l'espace des solutions).

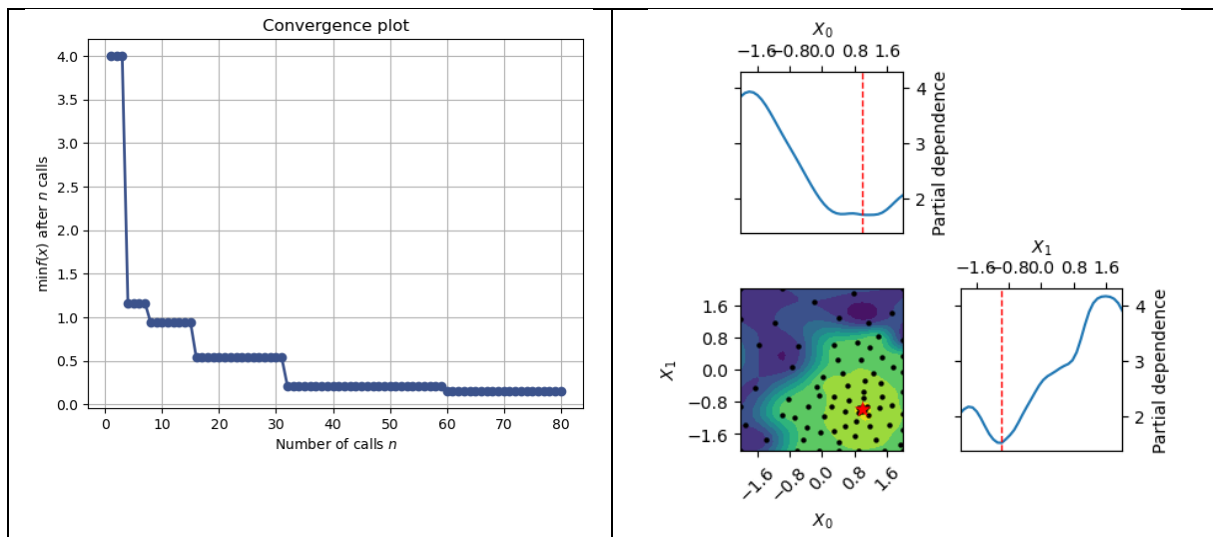


Figure 6 – Résultat algorithme BO : convergence (à gauche) et affichage de la fonction estimée en 2D (à droite).